

3 November 2023

Greedy Algorithms

* Beware! When using a greedy algo, need to prove optimality.

- often, easy to implement b/c just using "current" info, forgetting past/future

- proofs often have the flavor.

Let A be the greedy ans.

Let B be an optimal sol'n.

[Prove A is at least as good as B].

Today: Huffman codes.

Def'n

binary code: string of 1's and 0's
(typically encoding something)

prefix-free code: a "code" such that no code is a prefix of another.

length of an encoded message (using binary code)

$$\sum_{i=1}^n \text{freq}(i) \cdot \text{depth}(i)$$

frequency of
in letter

depth of letter in
binary tree

goal:
minimize this!

①

MORSE CODE

A ● -	J ● - - -	S ● ● ● ●
B - ● ● ●	K - ● -	T -
C - ● - ●	L ● - ● ●	U ● ● ●
D - ● ●	M - -	V ● ● ●
E ●	N - ●	W ● -
F ● ● - ●	O - - -	X - ● ●
G - - ●	P ● - - ●	Y - ● -
H ● ● ● ●	Q - - ● -	Z - - ●
I ● ●	R ● - ●	

some cut-off
↓

This is not a prefix-free code.

Proof: Consider A (● -) and J (● - - -). A is the prefix to J. □

↑ proof by counter-example. There are more!

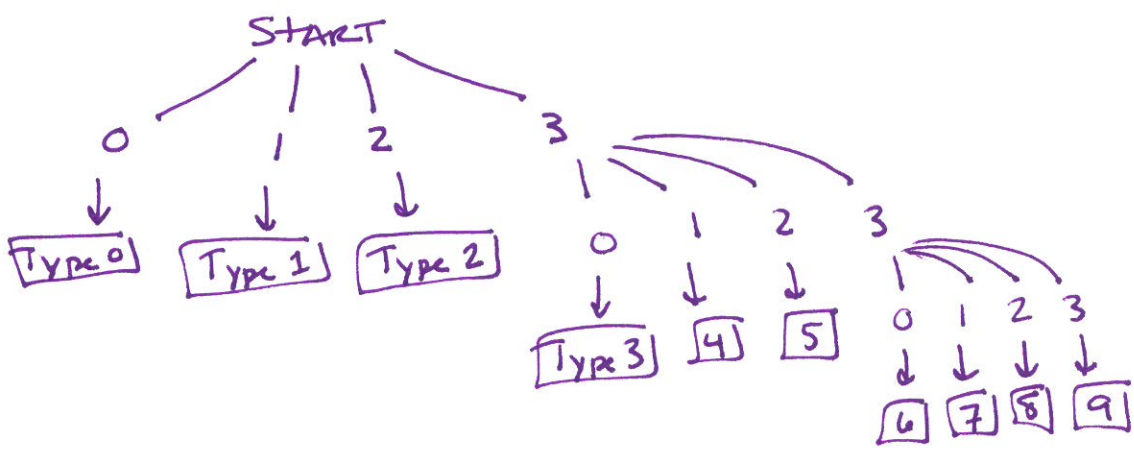
Can consider it as a binary code (e.g., ● = 1, - = 0), but b/c not prefix-free, messages can be confusing. So, also use WA

Table 1. Tapulator numeric codes.

Numbers	Code	Gestures
0	0	swipe
1	1	1-finger tap
2	2	2-finger tap
3	3 + 0	3-finger tap + swipe
4	3 + 1	3-finger tap + 1-finger tap
5	3 + 2	3-finger tap + 2-finger tap
6	3 + 3 + 0	2 × (3-finger tap) + swipe
7	3 + 3 + 1	2 × (3-finger tap) + 1-finger tap
8	3 + 3 + 2	2 × (3-finger tap) + 2-finger tap
9	3 + 3 + 3	3 × (3-fingers tap)

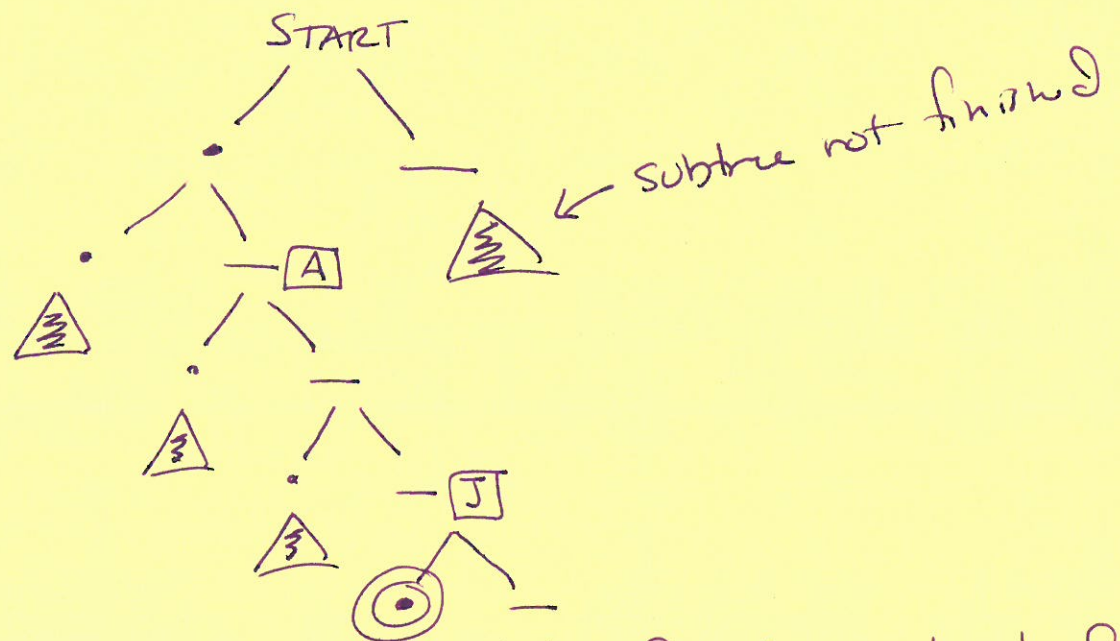
This is not a binary code .
Pf: There are 4 "letters" of the code. □

This is a prefix-free code.



3

What would our tree look like if not a prefix-free code?



The "prefix word" is now an internal node, not a leaf.
So, unclear if new letter ~~or new word~~ ^{starting or} still continuing w/ the letter

At ④, how did I get there? JE or AG?

In 1951, as a PhD student at MIT, David Huffman developed the following greedy algorithm to produce such an optimal code:⁴

HUFFMAN: Merge the two least frequent letters and recurse.

Huffman's algorithm is best illustrated through an example. Suppose we want to encode the following helpfully self-descriptive sentence, discovered by Lee Sallows:⁵

This sentence contains three a's, three c's, two d's, twenty-six e's, five f's, three g's, eight h's, thirteen i's, two l's, sixteen n's, nine o's, six r's, twenty-seven s's, twenty-two t's, two u's, five v's, eight w's, four x's, five y's, and only one z.

To keep things simple, let's ignore the forty-four spaces, nineteen apostrophes, nineteen commas, three hyphens, and only one period, and encode only the letters, as though the message were written in *scriptio continua*:

THISSENTENCECONTAINSTHREEASTHREECSTWODSTWENTYSIXESFIVEFST
HREEGSEIGHTHSTHIRTEENISTWOLSSIXTEENNSNINEOSSIXRSTWENTYSEV
ENSTWENTYTWOTSTWOUSFIVEVEIGHTWSFOURXS FIVEYSANDONLYONEZ⁶

Here is the frequency table for Sallows' sentence:

A	C	D	E	F	G	H	I	L	N	O	R	S	T	U	V	W	X	Y	Z
3	3	2	26	5	3	8	13	2	16	9	6	27	22	2	5	8	4	5	1

Huffman's algorithm picks out the two least frequent letters, breaking ties arbitrarily—in this case, say, Z and D—and merges them together into a single

⁴Huffman was a student in an information theory class taught by Robert Fano, who was a close colleague of Claude Shannon, the father of information theory. Fano and Shannon had previously developed a different greedy algorithm for producing prefix codes—split the frequency array into two subarrays as evenly as possible, and then recursively build a code for each subarray—but these Fano-Shannon codes were known not to be optimal. Fano posed the problem of finding an optimal prefix code to his class. Huffman decided to solve the problem as a class project, instead of taking a final exam, not realizing that the problem was open, or that Fano and Shannon had already tried and failed to solve it. After several months of fruitless effort, Huffman eventually gave up and decided to take the final exam after all. As he was throwing his notes in the trash, the solution dawned on him. Huffman would later describe the epiphany as “the absolute lightning of sudden realization”.

⁵This sentence was first reported by Alexander Dewdney in his October 1984 “Computer Recreations” column in *Scientific American*. Sallows himself published the remarkable story of its discovery in 1985, along with several other self-descriptive sentences; you can find Sallows' paper on his web site. Frustrated with the slow progress of his code running on a VAX 11/780, Sallows designed and built dedicated hardware to perform a brute-force search for self-descriptive sentences with various flavor text (“This pangram has...”, “This sentence contains exactly...”, and so on). Careful theoretical analysis limited the search space to just over six billion possibilities, which his 1-MHz Pangram Machine enumerated in just under two hours.

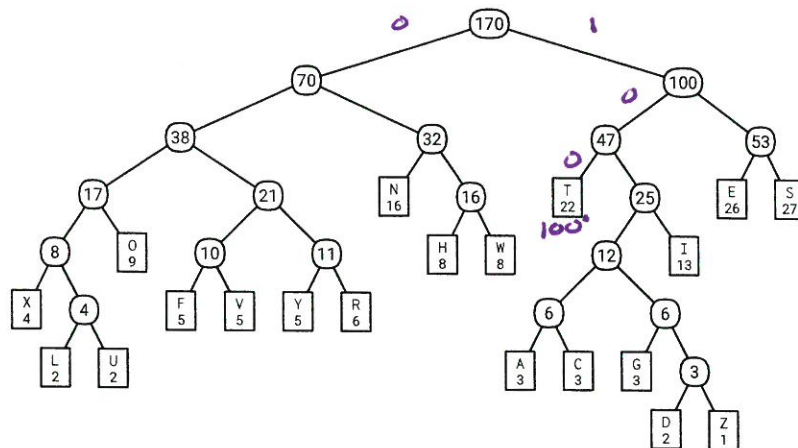
⁶... and he talked for forty-five minutes, and nobody understood a word that he said, but we had fun fillin' out the forms and playin' with the pencils on the bench there.

new character Σ with frequency 3. This new character becomes an internal node in the code tree we are constructing, with Z and D as its children; it doesn't matter which child is which. The algorithm then recursively constructs a Huffman code for the new frequency table

A	C	E	F	G	H	I	L	N	O	R	S	T	U	V	W	X	Y	Σ
3	3	26	5	3	8	13	2	16	9	6	27	22	2	5	8	4	5	3

D + Z are now merged to become a "new letter"

After 19 merges, all 20 letters have been merged together. The record of merges gives us our code tree. The algorithm makes a number of arbitrary choices; as a result, there are actually several different Huffman codes. One such Huffman code is shown below; numbers in non-leaf nodes are frequencies for merged characters. For example, the code for A is 101000, and the code for S is 111.



Encoding Sallows' sentence with this particular Huffman code would yield a bit string that starts like so:

100 0110 1011 111 111 110 010 100 110 010 101001 110 101001 0001 010 100 ...
T H I S S E N T E N C E C O N T

Here is the list of costs for encoding each character in Sallows' sentence, along with that character's contribution to the total length of the encoded sentence:

char	A	C	D	E	F	G	H	I	L	N	O	R	S	T	U	V	W	X	Y	Z
freq	3	3	2	26	5	3	8	13	2	16	9	6	27	22	2	5	8	4	5	1
depth	6	6	7	3	5	6	4	4	6	3	4	5	3	3	6	5	4	5	5	7
total	18	18	14	78	25	18	32	52	12	48	36	30	81	66	12	25	32	20	25	7

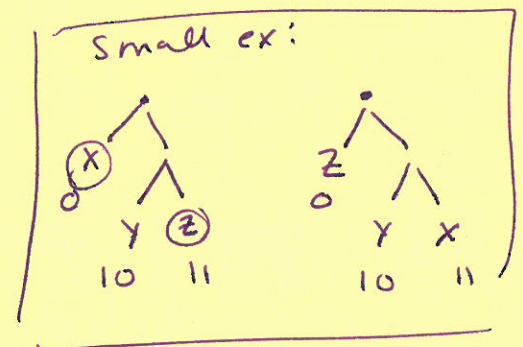
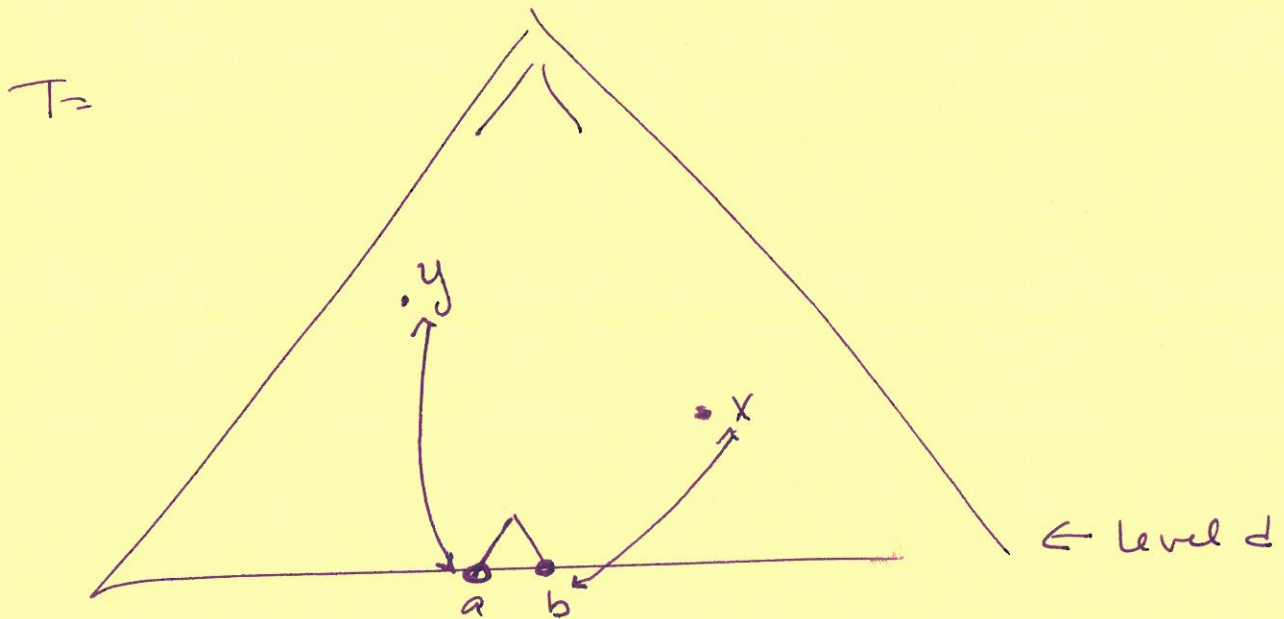
Altogether, the encoded message is 649 bits long. Different Huffman codes encode the same characters differently, possibly with code words of different length, but the overall length of the encoded message is the same for every Huffman code: 649 bits.

Q: How do we know this is the shortest that the message can be?

M is a message & we want an optimal binary code.

Lemma: Let x and y be the two least frequently used characters in the message we want to encode. There exists a code tree in which x and y are siblings (and have the largest depth).

Proof: Let T be an optimal code tree.
Let d be the depth of T .



recursive subproblem, define $f[n+1] = f[1] + f[2]$. Our earlier exchange argument implies that 1 and 2 are (deepest) siblings in some optimal code for $f[1..n]$.

Let T' be the Huffman tree for $f[3..n+1]$; the inductive hypothesis implies that T' is an optimal code tree for the smaller set of frequencies. To obtain the final code tree T , we replace the leaf labeled $n+1$ with an internal node with two children, labelled 1 and 2. I claim that T is optimal for the original frequency array $f[1..n]$.

To prove this claim, we can express the cost of T in terms of the cost of T' as follows. (In these equations, $\text{depth}(i)$ denotes the depth of the leaf labelled i in either T or T' ; each leaf that appears in both T and T' has the same depth in both trees.)

$$\begin{aligned}
 \text{cost}(T) &= \sum_{i=1}^n f[i] \cdot \text{depth}(i) \\
 &= \sum_{i=3}^{n+1} f[i] \cdot \text{depth}(i) + f[1] \cdot \text{depth}(1) + f[2] \cdot \text{depth}(2) \\
 &\quad - f[n+1] \cdot \text{depth}(n+1) \\
 &= \text{cost}(T') + (f[1] + f[2]) \cdot \text{depth}(T) - f[n+1] \cdot (\text{depth}(T) - 1) \\
 &= \text{cost}(T') + f[1] + f[2] + (f[1] + f[2] - f[n+1]) \cdot (\text{depth}(T) - 1) \\
 &= \text{cost}(T') + f[1] + f[2]
 \end{aligned}$$

This equation implies that minimizing the cost of T is equivalent to minimizing the cost of T' ; in particular, attaching leaves labeled 1 and 2 to the leaf in T' labeled $n+1$ gives an optimal code tree for the original frequencies. $\square$

To efficiently construct a Huffman code, we keep the characters in a priority queue, using the character frequencies as priorities. We can represent the code tree as three arrays of indices, listing the Left and Right children and the Parent of each node. The leaves of the final code tree are nodes at indices 1 through n , and the root is the node with index $2n-1$. Pseudocode for the algorithm is shown in Figure 4.4. BUILDHUFFMAN performs $O(n)$ priority-queue operations: exactly $2n-1$ INSERTS and $2n-2$ EXTRACTMINS. If we implement the priority queue as a standard binary heap, each of these operations requires $O(\log n)$ time, and thus the entire algorithm runs in $O(n \log n)$ time.

Finally, simple algorithms to encode and decode messages using a fixed Huffman code are shown in Figure 4.5; both algorithms run in $O(m)$ time, where m is the length of the encoded message.